

Hi,

In this document you'll find some tips on how to implement various parts announced in the homework.

Exercise 1 - events detection

Scala/Java

1. First, create a new Kafka input DataFrame:

```
val dataFrame = sparkSession.readStream
  .format("kafka")
  .options(KafkaInputConnectionOptions)
  .load()
```

2. Define the JSON schema, for example with the StructType API:

```
val schemaForFiltering = StructType(Array(
  StructField("eventId", StringType),
  StructField("os", StringType),
  StructField("visitedPage", StringType)
))
```

3. Use from_json function to convert the input array of bytes into a struct:

```
functions.from_json($"value".cast("string"), schemaForFiltering)
```

Thanks to that, you can now access the os attribute and apply the filtering logic.

4. To add the column with the processing time, use the withColumn method

Python

1. First, create a new Kafka input DataFrame:

```
input_data_stream = spark.readStream \
  .option("kafka.bootstrap.servers", "localhost:29092") \
  .option("subscribe", "visits") \
  .option("startingOffsets", "EARLIEST") \
  .format("kafka") \
  .load()
```

2. Define the JSON schema, for example with the StructType API:

```
schema_for_filtering = StructType([
  StructField("eventId", StringType()),
  StructField("os", StringType()),
  StructField("visitedPage", StringType())
])
```

3. Use from_json function to convert the input array of bytes into a struct:

```
F.from_json(F.col("value").cast("string"), schema_for_filtering)
```

Thanks to that, you can now access the os attribute and apply the filtering logic.

4. To add the column with the processing time, use the withColumn method

Exercise 2 - data reprocessing

Scala/Java

1. In the reprocessing job use the input arguments to get the time range for reprocessing:

```
object ReprocessingJob {

  def main(args: Array[String]): Unit = {
    val firstDate = LocalDate.parse(args(0))
    val lastDate = LocalDate.parse(args(1))
  }
}
```

2. To read all the partitions you can use map with reduce functions. The former creates a list of DataFrames to reprocess:

```
val inputDataFrames = partitionDates.map(eventTime => {
  sparkSession.read.schema("key STRING, value
  STRING").json(s"${FilesOutputDir}/event_time=${eventTime}")
})
```

While the latter combines all them into a single DataFrame with the help of union operator:

```
val allDataToReprocess = inputDataFrames
  .reduce((partitionsSoFar, newPartition) => partitionsSoFar.union(newPartition))
```

3. To keep the single data processing logic you can simply isolate this part into a dedicated function.

Python

1. In the reprocessing job use the input arguments to get the time range for reprocessing:

```
def reprocess_data_between_dates(start_date: datetime, end_date: datetime):  
    # ...  
  
if __name__ == '__main__':  
    start_date = datetime.strptime(sys.argv[1], '%Y-%m-%d')  
    end_date = datetime.strptime(sys.argv[2], '%Y-%m-%d')  
    reprocess_data_between_dates(start_date, end_date)
```

2. To read all the reprocessed data you can create the list of DataFrames for each partition and combine them into a single DataFrame with the reduce function:

```
partition_dataframes_to_reprocess = []  
current_date = start_date  
while current_date <= end_date:  
    event_time = str(current_date.strftime('%Y-%m-%d'))  
    partition_dataframes_to_reprocess.append(  
        spark.read.schema("key STRING, value STRING")  
            .json(f"{get_files_output_dir()}/event_time={event_time}")  
    )  
    current_date += timedelta(days=1)  
  
final_dataframe_to_reprocess = functools.reduce(  
    lambda all_partitions, partition_df: all_partitions.union(partition_df),  
    partition_dataframes_to_reprocess)
```

3. To keep the single data processing logic you can simply isolate this part into a dedicated function.

Exercise 3 - Dead-Letter Queue

Scala/Java

1. Create a case class with 3 attributes: key, value, and topic, for example:

```
case class VisitPageMappingResult(value: String, key: String, topic: String)
```

2. Change the map function:

- wrap the failing code with a try-catch block
- return the class created in the previous point that:
 - for a successful execution, inside the try part, should have the topic parameter set to valid_data
 - for a failure, inside the catch part, should have the topic parameter set to dead_letter

Python

1. Change the `get_visited_page` function:

- wrap the failing code with a try-catch block
- add an extra attribute to the returned dictionary; it should be called **topic**
 - the `topic` attribute should be set to:
 - `valid_data` for a successful execution
 - `dead_letter` for a failure

Exercise 4 - backpressure

Scala/Java

1. Define the input reading part:

```
case class RowToProcess(id: Int, row_value: String, generation_time: Timestamp)

val inputStream = sparkSession.readStream.format("kafka")
  .options(Map(
    "kafka.bootstrap.servers" -> "localhost:29092",
    "subscribe" -> "data_for_backpressure",
    "startingOffsets" -> "EARLIEST"
  )).load()

val rowToProcessSchema =
  ScalaReflection.schemaFor[RowToProcess].dataType.asInstanceOf[StructType]

val rowsToProcess = inputStream
  .select(functions.from_json($"value".cast("string"),
    rowToProcessSchema).as("value"))
  .selectExpr("value.*")
  .as[RowToProcess]
```

2. Use `foreachBatch` sink and define a custom function with the backpressure management.

3. The function should save backpressure files with the `batchNumber` and load those files before writing the data:

```
case class RowToProcess(id: Int, row_value: String, generation_time: Timestamp)

val inputStream = sparkSession.readStream.format("kafka")
  .options(Map(
    "kafka.bootstrap.servers" -> "localhost:29092",
    "subscribe" -> "data_for_backpressure",
    "startingOffsets" -> "EARLIEST"
  )).load()
```

```

val maybeBackpressuredDataset = if (batchNumber > 0) {
  Some(sparkSession.read.schema(rowToProcessSchema).json(s"${BackpressureDir}/${batchNumber - 1}")
    .as[RowToProcess])
} else {
  None
}
val datasetToProcess = maybeBackpressuredDataset
  .map(backpressuredDataset => backpressuredDataset.union(realTimeDataset))
  .getOrElse(realTimeDataset)

```

4. The function should apply next the window function to classify each record as "to process now" or "to process in the next micro-batch":

```

val maxRecordsPerJob = 5
val backpressureWindowSpec =
  Window.partitionBy("id").orderBy($"generation_time".asc)
val datasetWithBackpressureFlag = datasetToProcess.withColumn("is_backpressured",
  functions.when(
    functions.row_number().over(backpressureWindowSpec) > maxRecordsPerJob, true
  ).otherwise(false))

```

5. You should later write some code to:

- write all rows where `is_backpressured == true` to the backpressure store (`s"${BackpressureDir}/${batchNumber}"`)
- write remaining rows to the final sink

Python

1. Define the input reading part:

```

input_data_stream = spark.readStream \
  .option("kafka.bootstrap.servers", "localhost:29092") \
  .option("subscribe", "data_for_backpressure") \
  .option("startingOffsets", "EARLIEST") \
  .format("kafka") \
  .load()

json_schema = StructType([
  StructField("id", IntegerType()), StructField("row_value", StringType()),
  StructField("generation_time", TimestampType())
])

rows_to_process = input_data_stream.withColumn("filtering_struct",
  F.from_json(F.col("value").cast("string"), json_schema)) \
  .selectExpr("filtering_struct.*")

```

2. Use `foreachBatch` sink and define a custom function with the backpressure management.

3. The function should save backpressure files with the batchNumber and load those files before writing the data:

```
backpressure_dataset = None
if batch_version > 0:
    backpressure_dataset =
spark.read.schema(json_schema).json(f'{backpressure_dir}/{batch_version - 1}')

dataset_to_process = data_frame if not backpressure_dataset else
backpressure_dataset.union(data_frame)
```

4. The function should apply next the window function to classify each record as "to process now" or "to process in the next micro-batch":

```
backpressure_window_spec =
Window.partitionBy("id").orderBy(F.asc("generation_time"))
dataset_to_process_with_backpressure_flag = dataset_to_process \
    .withColumn("is_backpressured", F.when(
        F.row_number().over(backpressure_window_spec) > max_records_per_job,
        True).otherwise(False))
```

5. You should later write some code to:

- write all rows where is_backpressured == true to the backpressure store (\$"{BackpressureDir}/{batchNumber}")
- write remaining rows to the final sink

Hope it helps 🙏

Bartosz.